- Write a program to display " Hello World" using: Console.log() document.write( ) alert ()

```
<!DOCTYPE html>
<html>
<head>
  <title>Practical No. 1</title>
</head>
<body>

<script>
  // Display in console
  console.log("Hello World");

  // Display on webpage
  document.write("Hello World");

  // Display in alert box
  alert("Hello World");
</script>

</body>
</html>
```

Descript where is console located in browser.

The **console** is located in the browser's **Developer Tools (DevTools)** section.

It is used to display JavaScript output, errors, warning messages, and debugging information.

Method 2: Using Right Click

1. Open the webpage in the browser.
2. **Right-click** anywhere on the webpage.
3. Select **Inspect** from the menu.
4. The **Developer Tools** window will open.
5. Click on the **Console** tab.

- Write a program to display "Welcome " using Python script

```
<!DOCTYPE html>
<html>
<head>
  <title>Practical No. 2</title>
  <link rel="stylesheet" href="https://pyscript.net/releases/2025.8.1/core.css" />
  <script type="module" src="https://pyscript.net/releases/2025.8.1/core.js"></script>
</head>
<body>

  <script type="mpy" terminal>
    print("Welcome")
  </script>

</body>
</html>
```

- Create objects for the given problem with JSON:

Setup Angular development environment using:

Show

- Installation of Node.js and npm
- Installation of Angular CLI

Steps to Setup Angular Development Environment

1. Install Node.js and npm

1. Open the official Node.js website.
2. Download the **LTS version (.msi installer)**.
3. Double-click the downloaded file to run the installer.
4. Follow the setup wizard and click:

   Next → Accept License → Install → Finish
5. After installation, open **Command Prompt / VS Code Terminal**.
6. Verify the installation by typing:

node -v

npm -v

If version numbers are displayed, Node.js and npm are installed successfully.

---

2. Install Angular CLI

Open the terminal and type the following command:

npm install -g @angular/cli

Wait for the installation to complete.

---

3. Verify Angular CLI

Type the following command:

ng version

This displays:

- Angular CLI version
- Node version
- npm version

---

4. Create a New Angular Project

Use the command:

ng new my-app

Press **Enter** and follow the instructions.

---

5. Open Project Folder

Move into the created project folder:

cd my-app

---

6. Run the Angular Application

Start the local server using:

ng serve

---

7. Open in Browser

Type the following in browser:

http://localhost:4200

The default Angular welcome page will be displayed.

- Create objects for the given problem with JSON : Write a program to display "Good Morning" Message on web page

```
<!DOCTYPE html>

<html>

<head>

  <title>JSON Good Morning</title>

</head>

<body>



  <h2>Message Display</h2>
```

```
  <script>
    const messageObj = {
      "message": "Good Morning",

    };

    document.write(messageObj.message);
  </script>

</body>
</html>
```

- Write AngularJS program to design form using various controls and apply validations on input

```
<!DOCTYPE html>
<html ng-app="">
<head>
  <title>Form Validation</title>
  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body>
```

```
<form name="myForm">

  Name:
  <input type="text" name="name" ng-model="name" required>
  <span ng-show="myForm.name.$touched && myForm.name.$invalid">
    Name is required
  </span>
  <br><br>

  Email:
  <input type="email" name="email" ng-model="email" required>
  <span ng-show="myForm.email.$touched && myForm.email.$invalid">
    Enter valid email
  </span>
  <br><br>

  Age:
  <input type="number" name="age" ng-model="age" min="18" max="60" required>
  <span ng-show="myForm.age.$touched && myForm.age.$invalid">
    Enter age between 18 and 60
  </span>
  <br><br>

  <input type="submit" value="Submit" ng-disabled="myForm.$invalid">

</form>
```

```
</body>
</html>
```

- Write a program to display data model view and display data for given problem

```
<!DOCTYPE html>
<html ng-app="">
<head>
  <title>Model View Example</title>
  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body>

  <h2>Student Information</h2>

  <div ng-init="name='Om Nandre'; branch='Computer Engineering'; year='TYCO'">
    <p><b>Name:</b> {{name}}</p>
    <p><b>Branch:</b> {{branch}}</p>
    <p><b>Year:</b> {{year}}</p>
  </div>

</body>
</html>
```

- Write a program to implement different filters in AngularJS

```
<!DOCTYPE html>
```

```
<html ng-app="">
<head>
   <title>AngularJS Filters Example</title>
   <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body>


   <h2>AngularJS Filters Example</h2>


   <div ng-init="name='om nandre'; amount=5000; today=Date.now()">


      <p><b>Original Name:</b> {{name}}</p>
      <p><b>Uppercase Filter:</b> {{name | uppercase}}</p>
      <p><b>Lowercase Filter:</b> {{name | lowercase}}</p>


      <p><b>Currency Filter:</b> {{amount | currency:'₹'}}</p>


      <p><b>Date Filter:</b> {{today | date:'dd/MM/yyyy'}}</p>


   </div>
```

```
</body>

</html>
```

- Write a program to implement different events in Angular JS

```
<!DOCTYPE html>

<html ng-app="">

<head>

  <title>AngularJS Events Example</title>

  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>

</head>

<body>



  <h2>AngularJS Mouse and Key Events</h2>



  <!-- Mouse Events -->

  <div style="width:300px; height:120px; border:2px solid black; padding:10px;"

      ng-mousedown="msg='Mouse Button Pressed'"

      ng-mouseenter="msg='Mouse Entered Box'"

      ng-mouseleave="msg='Mouse Left Box'"

      ng-mousemove="msg='Mouse Moving Inside Box'"
```

```
        ng-mouseover="msg='Mouse Over Box'"

        ng-mouseup="msg='Mouse Button Released'">

       Move mouse inside this box

    </div>



    <p><b>Mouse Event:</b> {{msg}}</p>



    <hr>



    <!-- Key Events -->

    <p>Type in the box below:</p>

    <input type="text"

          ng-keydown="keymsg='Key Down Event'"

          ng-keypress="keymsg='Key Press Event'"

          ng-keyup="keymsg='Key Up Event'">



    <p><b>Key Event:</b> {{keymsg}}</p>



</body>

</html>
```

- Write a program displaying data in a table

```
<!DOCTYPE html>

<html ng-app="myApp">

<head>

  <title>Display Data in Table</title>

  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>

  <style>

    table {

      width: 60%;

      border-collapse: collapse;

      margin-top: 20px;

    }

    th, td {

      border: 1px solid black;

      padding: 10px;

      text-align: center;

    }

    th {

      background-color: lightgray;

    }
```

```
    </style>
</head>
<body ng-controller="tableCtrl">


    <h2>Student Data Table</h2>


    <table>
        <tr>
            <th>Roll No</th>
            <th>Name</th>
            <th>Branch</th>
            <th>Class</th>
        </tr>


        <tr ng-repeat="student in students">
            <td>{{student.roll}}</td>
            <td>{{student.name}}</td>
            <td>{{student.branch}}</td>
            <td>{{student.class}}</td>
        </tr>
```

```
  </table>


  <script>
    var app = angular.module("myApp", []);


    app.controller("tableCtrl", function($scope) {
      $scope.students = [
        {roll: 1, name: "Om", branch: "CO", class: "TYCO"},
        {roll: 2, name: "Rahul", branch: "CO", class: "TYCO"},
        {roll: 3, name: "Priya", branch: "CO", class: "TYCO"},
        {roll: 4, name: "Amit", branch: "CO", class: "TYCO"}
      ];
    });
  </script>


</body>
</html>
```

- Write a program to implement CSS to table data-odd and even rows

```
<!DOCTYPE html>
```

```
<html ng-app="myApp">
<head>
  <title>Odd and Even Row Styling</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>


  <style>
    table {
      width: 60%;
      border-collapse: collapse;
      margin-top: 20px;
    }


    th, td {
      border: 1px solid black;
      padding: 10px;
      text-align: center;
    }


    th {
      background-color: lightgray;
```

```
        }

        .odd-row {
            background-color: #f2f2f2;
        }

        .even-row {
            background-color: #d9edf7;
        }
    </style>
</head>
<body ng-controller="tableCtrl">

    <h2>Student Table with Odd and Even Row CSS</h2>

    <table>
        <tr>
            <th>Roll No</th>
            <th>Name</th>
            <th>Branch</th>
```

```
    </tr>


    <tr ng-repeat="student in students"
        ng-class-odd="'odd-row'"
        ng-class-even="'even-row'">


        <td>{{student.roll}}</td>
        <td>{{student.name}}</td>
        <td>{{student.branch}}</td>
    </tr>
</table>


<script>
    var app = angular.module("myApp", []);


    app.controller("tableCtrl", function($scope) {
        $scope.students = [
            {roll: 1, name: "Om", branch: "CO"},
            {roll: 2, name: "Rahul", branch: "CO"},
            {roll: 3, name: "Priya", branch: "CO"},
```

```
            {roll: 4, name: "Amit", branch: "CO"},

            {roll: 5, name: "Sneha", branch: "CO"}

          ];

      });

   </script>



</body>

</html>
```

- Write programs for implementation of different methods of AngularJS Controllers

```
<!DOCTYPE html>

<html ng-app="myApp">

<head>

   <title>AngularJS Controllers Methods</title>

   <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>

</head>

<body>



   <h2>Different Methods of AngularJS Controllers</h2>
```

```
<!-- Basic Controller -->
<div ng-controller="BasicCtrl">
    <h3>Basic Controller</h3>
    <p>{{message}}</p>
    <button ng-click="greet()">Click Me</button>
</div>


<hr>


<!-- Nested Controllers -->
<div ng-controller="ParentCtrl">
    <h3>Parent Controller</h3>
    <p>{{parentData}}</p>


    <div ng-controller="ChildCtrl">
        <h3>Child Controller</h3>
        <p>{{parentData}}</p>
        <p>{{childData}}</p>
    </div>
</div>
```

```
<hr>


<!-- Filter in Controller -->
<div ng-controller="FilterCtrl">
    <h3>Filter in Controller</h3>
    <p>Name: {{name | uppercase}}</p>
    <p>Salary: {{salary | currency:'₹'}}</p>
</div>


<script>
    var app = angular.module("myApp", []);


    // Basic Controller
    app.controller("BasicCtrl", function($scope) {
        $scope.message = "Hello! Welcome to AngularJS";


        $scope.greet = function() {
            alert("Welcome!");
        };
```

```
    });


    // Parent Controller
    app.controller("ParentCtrl", function($scope) {
        $scope.parentData = "This is Parent Controller Data";
    });


    // Child Controller
    app.controller("ChildCtrl", function($scope) {
        $scope.childData = "This is Child Controller Data";
    });


    // Filter Controller
    app.controller("FilterCtrl", function($scope) {
        $scope.name = "om nandre";
        $scope.salary = 50000;
    });
  </script>


</body>
```

```
</html>
```

- Write programs to demonstrate use of controllers in external files

Html file

```
<!DOCTYPE html>

<html ng-app="myApp">

<head>

  <title>External Controller Example</title>

  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>

  <script src="controller.js"></script>

</head>

<body ng-controller="SimpleController">



  <h2>AngularJS Controller in External File</h2>



  <p>{{message}}</p>



  <button ng-click="showMessage()">Click Me</button>



</body>

</html>
```

External JS File (controller.js)

```
angular.module('myApp', [])

.controller('SimpleController', function($scope) {



  $scope.message = "Hello from External Controller!";



  $scope.showMessage = function() {

    alert("Welcome to AngularJS External Controller");

  };



});
```

- Write a program to handle data using React form

```
<!DOCTYPE html>

<html>

<head>

  <title>React Form Example</title>


  <script src="https://unpkg.com/react@18/umd/react.development.js"></script>

  <script src="https://unpkg.com/react-dom@18/umd/react-

dom.development.js"></script>

  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

</head>
```

```
<body>

  <div id="root"></div>

  <script type="text/babel">
    function App() {
      const [name, setName] = React.useState("");
      const [submittedName, setSubmittedName] = React.useState("");

      const handleChange = (event) => {
        setName(event.target.value);
      };

      const handleSubmit = (event) => {
        event.preventDefault();
        setSubmittedName(name);
      };

      return (
        <div>
          <h2>React Form Handling</h2>

          <form onSubmit={handleSubmit}>
            <label>Enter Name: </label>
            <input
              type="text"
```

```
                    value={name}
                    onChange={handleChange}
                  />

                  <button type="submit">Submit</button>
                </form>

                <h3>Submitted Data:</h3>
                <p>{submittedName}</p>
              </div>
            );
          }

          ReactDOM.createRoot(document.getElementById("root")).render(<App />);
        </script>

      </body>
      </html>
```

- Write a program to pass function argument into React program and implement the life cycle of React

```
<!DOCTYPE html>
<html>
<head>
```

```
    <title>React Lifecycle Example</title>


    <script src="https://unpkg.com/react@18/umd/react.development.js"></script>

    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>

    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

</head>

<body>


    <div id="root"></div>


    <script type="text/babel">

        class App extends React.Component {

            constructor(props) {

                super(props);

                this.state = {

                    message: "Welcome",

                    count: 0

                };

                console.log("Constructor Called");

            }
```

```
componentDidMount() {
    console.log("Component Mounted");
}

componentDidUpdate() {
    console.log("Component Updated");
}

componentWillUnmount() {
    console.log("Component Unmounted");
}

showMessage(name) {
    alert("Hello " + name);
    this.setState({ count: this.state.count + 1 });
}

render() {
    return (
```

```
            <div>
              <h2>React Lifecycle Example</h2>
              <p>{this.state.message}</p>
              <p>Count: {this.state.count}</p>


              <button onClick={() => this.showMessage("Om")}>
                Click Me
              </button>
            </div>
          );
        }
      }

      ReactDOM.createRoot(document.getElementById("root")).render(<App />);
    </script>

  </body>
  </html>
```

- Write a program to implement states of React Hooks

```
<!DOCTYPE html>

<html>

<head>

    <title>React useState Hook Example</title>



    <script src="https://unpkg.com/react@18/umd/react.development.js"></script>

    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>

    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

</head>

<body>



    <div id="root"></div>



    <script type="text/babel">

        function App() {

            const [count, setCount] = React.useState(0);



            const increment = () => {

                setCount(count + 1);
```

```
    };

    const decrement = () => {
      setCount(count - 1);
    };

    return (
      <div>
        <h2>React useState Hook Example</h2>
        <p>Count: {count}</p>

        <button onClick={increment}>Increase</button>
        <button onClick={decrement}>Decrease</button>
      </div>
    );
  }

  ReactDOM.createRoot(document.getElementById("root")).render(<App />);
</script>
```

</body>

</html>

- Write the basic steps of your microproject developed.

Basic Steps of Microproject Developed

1. **Problem Identification**

   First, the problem and objective of the project were identified.

   The aim was to develop a platform where users can share, explore, and manage innovative project ideas.

2. **Requirement Analysis**

   The required features were listed, such as:

   - user registration/login
   - project idea submission
   - viewing ideas
   - search/filter options
   - responsive interface

3. **Planning and Design**

   The structure of the project was planned.

   Wireframes / rough layout for pages like Home, Login, Dashboard, and Project Submission page were prepared.

4. **Technology Selection**

   Technologies and tools were selected, such as:

   - HTML

- CSS
- JavaScript
- React / AngularJS
- Database (if used)

5. **Frontend Development**

   The user interface pages were created with proper styling and navigation.

6. **Backend / Logic Implementation**

   Functions for handling data, form validation, login, and storing project information were implemented.

7. **Testing**

   The project was tested for:

   - functionality
   - input validation
   - UI responsiveness
   - error handling

8. **Debugging and Improvements**

   Errors were fixed and design improvements were made for better user experience.

9. **Final Deployment / Execution**

   The final project was executed in browser / local server and verified.

10. **Documentation**

    Project report, screenshots, test cases, and conclusion were prepared.